

ISPGAYA
instituto superior politécnico

Escola Superior de Ciência e Tecnologia

Licenciatura em Engenharia Informática - 2024/2025

Linguagens e Paradigmas de Programação



```

1 def verificar_dados_alunos():
2     """
3     Solicita o nome de um aluno e as suas notas, verificando se as notas são válidas (entre 0 e 20).
4     Armazena os dados num dicionário e exibe as informações verificadas.
5     """
6     nome_aluno = input("Digite o nome do aluno: ") # Pede ao utilizador o nome do aluno
7     notas = [] # Cria uma lista para armazenar as notas
8
9     while True: # Inicia um ciclo infinito para inserir várias notas
10         try:
11             nota_str = input("Digite uma nota do aluno (ou 'fim' para terminar): ") # Pede ao utilizador uma nota
12             if nota_str.lower() == 'fim': # Se o utilizador escrever 'fim', termina o ciclo
13                 break
14             nota = float(nota_str) # Tenta converter nota para float
15             if 0 <= nota <= 20: # Verifica se nota está no intervalo permitido
16                 notas.append(nota) # Adiciona nota à lista
17             else:
18                 print("Erro: A nota deve estar entre 0 e 20.") # Mensagem de erro se nota for inválida
19         except ValueError:
20             print("Erro: Por favor, digite um número válido para a nota.") # Mensagem de erro se não for possível converter para número
21
22     dados_aluno = {
23         'nome': nome_aluno, # Guarda o nome do aluno
24         'notas': notas # Guarda a lista de notas
25     }
26
27     print("\nDados do aluno verificados:")
28     print(f"Nome: {dados_aluno['nome']}") # Mostra o nome do aluno
29     if dados_aluno['notas']: # Se existirem notas inseridas
30         print(f"Notas: {', '.join(map(str, dados_aluno['notas']))}") # Mostra a lista de notas
31         media = sum(dados_aluno['notas']) / len(dados_aluno['notas']) # Calcula a média das notas
32         print(f"Média: {media:.2f}") # Mostra a média com duas casas decimais
33     else:
34         print("Nenhuma nota foi inserida para este aluno.") # Mensagem caso não tenham sido inseridas notas
35
36 # Executar o algoritmo
37 verificar_dados_alunos()

```

Aluno:

Miguel Magalhães (2021103166)

Docente: Sr. Prof. José Arnaud

15 de Maio de 2025

ÍNDICE DE CONTEÚDOS

Índice de Conteúdos.....	2
Índice de Imagens	3
Introdução	5
Enquadramento	6
Objetivos	7
Estrutura do Relatório	8
Desenvolvimento	9
Indicar o enunciado	9
Comentário das Linhas de Código Mais Relevantes.....	10
Implementação em Java — Código Completo Comentado.....	10
Implementação em Java — Excertos mais relevantes	11
Implementação em Python – Código Completo Comentado.....	13
Implementação em Python – Excertos mais relevantes	13
Reflexão sobre as Principais Diferenças entre o Código em Python e em Java	15
Limitações do Algoritmo	18
Melhorias que Podiam ser Implementadas	20
Conclusão.....	22
Anexos	23

ÍNDICE DE IMAGENS

Figura i.: Print do código teste.java comentado.....	10
Figura ii.: Criação de um objeto Scanner para leitura de dados introduzidos pelo utilizador através do teclado.	11
Figura iii.: Solicita e lê do utilizador o nome do aluno, armazenando-o na variável nomeAluno.	11
Figura iv.: Cria uma lista dinâmica de valores decimais (double) para armazenar as notas do aluno.....	11
Figura v.: Inicia um ciclo infinito que permite a inserção sucessiva de notas até ser indicado o fim pelo utilizador.....	11
Figura vi.: Condição de paragem do ciclo: termina a introdução de notas caso o utilizador escreva "fim" (sem sensibilidade a maiúsculas/minúsculas).	11
Figura vii.: Conversão da string lida para um valor decimal (double) e validação do intervalo permitido. Se a nota for válida, é adicionada à lista de notas.....	12
Figura viii.: Mensagem de erro apresentada ao utilizador caso a nota não esteja no intervalo permitido.	12
Figura ix.: Tratamento de exceção: caso a conversão da nota falhe (caracteres não numéricos), informa o utilizador do erro.....	12
Figura x.: Verifica se foram introduzidas notas válidas para o aluno e, se existirem notas, calcula a soma e a média, apresentando-a formatada com duas casas decimais..	12
Figura xi.: Fecha o objeto Scanner e liberta os recursos associados:	13
Figura xii.: Print do código act.py comentado.....	13
Figura xiii.: Solicita e lê do utilizador o nome do aluno, armazenando-o na variável nome_aluno e inicializa uma lista vazia destinada ao armazenamento das notas inseridas.	13
Figura xiv.: Inicia um ciclo infinito que permitirá ao utilizador inserir múltiplas notas, terminando apenas quando for digitado "fim".....	14
Figura xv.: Solicita a nota ao utilizador e verifica se o comando "fim" foi digitado (ignorando maiúsculas/minúsculas) para terminar o ciclo.	14
Figura xvi.: Conversão da string para valor decimal (float) e validação do intervalo permitido. Se a nota for válida, é adicionada à lista de notas; caso contrário, é apresentada uma mensagem de erro.	14

Figura xvii.: Tratamento de exceção: caso a conversão falhe (por exemplo, caracteres não numéricos), apresenta uma mensagem de erro ao utilizador. 14

Figura xviii.: Se existirem notas válidas, apresenta a lista das notas e calcula a média (com duas casas decimais) e se não forem inseridas notas, informa o utilizador..... 15

INTRODUÇÃO

O presente relatório foi elaborado no âmbito da unidade curricular de Linguagens e Paradigmas de Programação, integrada na Licenciatura em Engenharia Informática. O objetivo fundamental deste trabalho consiste na análise crítica e comparativa de um algoritmo implementado em duas linguagens de programação distintas: Java e Python.

Neste contexto, pretende-se evidenciar não apenas as diferenças sintáticas e estruturais entre as linguagens, mas também a abordagem ao paradigma de programação e à resolução de problemas. O trabalho inclui, ainda, a identificação das linhas de código mais relevantes, a reflexão sobre as limitações do algoritmo proposto e a apresentação de sugestões de melhoria.

A realização deste exercício pretende consolidar os conhecimentos adquiridos na unidade curricular, promovendo o desenvolvimento de competências analíticas e críticas relativamente às opções de implementação e à utilização adequada das ferramentas proporcionadas por diferentes linguagens de programação. Adicionalmente, visa-se incentivar a reflexão sobre as boas práticas de programação e a adaptabilidade dos algoritmos às necessidades do utilizador.

ENQUADRAMENTO

A aprendizagem e compreensão das diferentes linguagens de programação constituem um pilar fundamental na formação em Engenharia Informática. A disciplina de Linguagens e Paradigmas de Programação propõe, por isso, a exploração prática das principais linguagens utilizadas na indústria, com especial enfoque nas suas particularidades, vantagens e limitações.

O exercício proposto insere-se neste contexto pedagógico, permitindo ao estudante contactar diretamente com a implementação de algoritmos em linguagens de alto nível, nomeadamente Java e Python. Ambas as linguagens são amplamente adotadas, apresentando diferentes filosofias de desenvolvimento, modelos de gestão de memória, estruturas de controlo e mecanismos de tratamento de erros.

Ao analisar a resolução de um mesmo problema em duas linguagens, o estudante é desafiado a compreender não só as distinções técnicas, mas também as razões subjacentes a determinadas escolhas de implementação. Esta abordagem fomenta a versatilidade, a capacidade de adaptação a diferentes ambientes de programação e a aquisição de competências essenciais para o desenvolvimento de soluções robustas e eficientes.

OBJETIVOS

O principal objetivo deste trabalho consiste em promover uma análise detalhada e fundamentada da implementação de um algoritmo simples, utilizando as linguagens de programação Java e Python. Pretende-se, assim, alcançar os seguintes objetivos específicos:

- Indicar o enunciado do algoritmo proposto.
- Comentar as linhas de código mais relevantes nas implementações em Java e Python.
- Fazer uma reflexão sobre as principais diferenças entre o código em Python e em Java.
- Identificar as limitações do algoritmo apresentado.
- Apontar melhorias que poderiam ser implementadas.

A concretização destes objetivos permite desenvolver uma análise crítica e comparativa entre as linguagens de programação, aprofundando a compreensão dos seus principais conceitos e práticas de implementação.

ESTRUTURA DO RELATÓRIO

O presente relatório encontra-se organizado da seguinte forma:

- **Introdução:** Apresenta o contexto e a motivação do trabalho.
- **Enquadramento:** Descreve o âmbito da análise e a sua relevância na formação em Engenharia Informática.
- **Objetivos:** Define, de forma clara, os principais objetivos a atingir com este trabalho.
- **Desenvolvimento:** Inclui o enunciado do algoritmo, os códigos em Java e Python comentados, a análise das linhas mais relevantes, a reflexão comparativa entre as linguagens, as limitações do algoritmo e propostas de melhoria.
- **Conclusão:** Resume os principais pontos analisados, evidenciando as aprendizagens e considerações finais.
- **Bibliografia:** Lista as fontes e referências consultadas.
- **Anexos:** Contém materiais complementares ou códigos na íntegra, caso aplicável.

DESENVOLVIMENTO

INDICAR O ENUNCIADO

O exercício em análise tem como finalidade a criação de um algoritmo capaz de registar o nome de um aluno e as respetivas notas, garantindo que cada nota inserida se encontra no intervalo válido de 0 a 20 valores. O programa deverá solicitar ao utilizador, de forma iterativa, a introdução das notas do aluno, permitindo a sua inserção até que o utilizador indique explicitamente a vontade de terminar (através do comando “fim”). Após a recolha dos dados, o algoritmo deverá apresentar o nome do aluno, a lista das notas inseridas e o cálculo da média aritmética das mesmas. Caso não sejam introduzidas notas válidas, o sistema deverá informar o utilizador desse facto.

De modo a proporcionar uma análise comparativa entre diferentes paradigmas e estilos de programação, o mesmo algoritmo foi desenvolvido em duas linguagens de programação distintas: Java e Python. Esta abordagem permite evidenciar diferenças ao nível da sintaxe, gestão de tipos de dados, estruturas de controlo e tratamento de exceções, mantendo inalterada a lógica subjacente à resolução do problema.

COMENTÁRIO DAS LINHAS DE CÓDIGO MAIS RELEVANTES

Nesta secção, apresenta-se, em primeiro lugar, o código completo de cada implementação (Java e Python), acompanhado de comentários explicativos, de modo a clarificar o funcionamento global do algoritmo.

A seguir, são destacados e analisados, de forma individual, os excertos considerados mais relevantes, justificando a sua importância para o correto funcionamento da solução desenvolvida.

IMPLEMENTAÇÃO EM JAVA — CÓDIGO COMPLETO COMENTADO

```
teste.java > ...
1  package com.mycompany.java.project;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Scanner;
6
7  public class teste {
8
9      Run | Debug
10     public static void main(String[] args) {
11         Scanner scanner = new Scanner(System.in); // Cria um objeto Scanner para ler a entrada do utilizador
12
13         System.out.print(s:"Digite o nome do aluno: ");
14         String nomeAluno = scanner.nextLine(); // Lê o nome do aluno inserido pelo utilizador
15
16         List<Double> notas = new ArrayList<>(); // Cria uma lista para armazenar as notas do aluno
17
18         while (true) { // Inicia um ciclo infinito para inserir várias notas
19             System.out.print(s:"Digite uma nota do aluno (ou 'fim' para terminar): ");
20             String notaStr = scanner.nextLine(); // Lê a nota inserida pelo utilizador como String
21
22             if (notaStr.equalsIgnoreCase(anotherString:"fim")) { // Se o utilizador escrever 'fim', termina o ciclo
23                 break;
24             }
25
26             try {
27                 double nota = Double.parseDouble(notaStr); // Tenta converter a nota para double
28                 if (nota >= 0 && nota <= 20) { // Verifica se a nota está no intervalo permitido
29                     notas.add(nota); // Adiciona a nota à lista
30                 } else {
31                     System.out.println(x:"Erro: A nota deve estar entre 0 e 20."); // Mensagem de erro se a nota for
32                                     // inválida
33                 }
34             } catch (NumberFormatException e) {
35                 System.out.println(x:"Erro: Por favor, digite um número válido para a nota."); // Mensagem de erro se não
36                                     // for possível converter
37                                     // para número
38             }
39         }
40
41         System.out.println(x:"\nDados do aluno verificados:");
42         System.out.println("Nome: " + nomeAluno); // Mostra o nome do aluno
43
44         if (!notas.isEmpty()) { // Se existirem notas inseridas
45             System.out.println("Notas: " + notas); // Mostra a lista de notas
46             double soma = 0;
47             for (double nota : notas) {
48                 soma += nota; // Calcula a soma das notas
49             }
50             double media = soma / notas.size(); // Calcula a média das notas
51             System.out.printf(format:"Média: %.2f\n", media); // Mostra a média com duas casas decimais
52         } else {
53             System.out.println(x:"Nenhuma nota foi inserida para este aluno."); // Mensagem caso não tenham sido inseridas
54                                     // notas
55         }
56
57         scanner.close(); // Fecha o Scanner para libertar recursos
58     }
59 }
```

Figura i.: Print do código teste.java comentado.

IMPLEMENTAÇÃO EM JAVA — EXCERTOS MAIS RELEVANTES

- Cria um objeto Scanner para leitura de dados introduzidos pelo utilizador através do teclado:

```
Scanner scanner = new Scanner(System.in); // Cria um objeto Scanner para ler a entrada do utilizador
```

Figura ii.: Criação de um objeto Scanner para leitura de dados introduzidos pelo utilizador através do teclado.

- Solicita e lê do utilizador o nome do aluno, armazenando-o na variável nomeAluno:

```
String nomeAluno = scanner.nextLine(); // Lê o nome do aluno inserido pelo utilizador
```

Figura iii.: Solicita e lê do utilizador o nome do aluno, armazenando-o na variável nomeAluno.

- Cria uma lista dinâmica de valores decimais (double) para armazenar as notas do aluno:

```
List<Double> notas = new ArrayList<>(); // Cria uma lista para armazenar as notas do aluno
```

Figura iv.: Cria uma lista dinâmica de valores decimais (double) para armazenar as notas do aluno.

- Inicia um ciclo infinito que permite a inserção sucessiva de notas até ser indicado o fim pelo utilizador:

```
while (true) { // Inicia um ciclo infinito para inserir várias notas  
    System.out.print(s:"Digite uma nota do aluno (ou 'fim' para terminar): ");  
    String notaStr = scanner.nextLine(); // Lê a nota inserida pelo utilizador como String
```

Figura v.: Inicia um ciclo infinito que permite a inserção sucessiva de notas até ser indicado o fim pelo utilizador.

- Condição de paragem do ciclo: termina a introdução de notas caso o utilizador escreva "fim" (sem sensibilidade a maiúsculas/minúsculas).

```
if (notaStr.equalsIgnoreCase(anotherString:"fim")) { // Se o utilizador escrever 'fim', termina o ciclo  
    break;  
}
```

Figura vi.: Condição de paragem do ciclo: termina a introdução de notas caso o utilizador escreva "fim" (sem sensibilidade a maiúsculas/minúsculas).

- Conversão da string lida para um valor decimal (double) e validação do intervalo permitido. Se a nota for válida, é adicionada à lista de notas:

```
double nota = Double.parseDouble(notaStr); // Tenta converter a nota para double
if (nota >= 0 && nota <= 20) { // Verifica se a nota está no intervalo permitido
    notas.add(nota); // Adiciona a nota à lista
}
```

Figura vii.: Conversão da string lida para um valor decimal (double) e validação do intervalo permitido. Se a nota for válida, é adicionada à lista de notas.

- Mensagem de erro apresentada ao utilizador caso a nota não esteja no intervalo permitido:

```
System.out.println(x:"Erro: A nota deve estar entre 0 e 20."); // Mensagem de erro se a nota for
// inválida
```

Figura viii.: Mensagem de erro apresentada ao utilizador caso a nota não esteja no intervalo permitido.

- Tratamento de exceção: caso a conversão da nota falhe (caracteres não numéricos), informa o utilizador do erro:

```
} catch (NumberFormatException e) {
    System.out.println(x:"Erro: Por favor, digite um número válido para a nota."); // Mensagem de erro se não
// for possível converter
// para número
}
```

Figura ix.: Tratamento de exceção: caso a conversão da nota falhe (caracteres não numéricos), informa o utilizador do erro.

- Verifica se foram introduzidas notas válidas para o aluno e, se existirem notas, calcula a soma e a média, apresentando-a formatada com duas casas decimais:

```
if (!notas.isEmpty()) { // Se existirem notas inseridas
    System.out.println("Notas: " + notas); // Mostra a lista de notas
    double soma = 0;
    for (double nota : notas) {
        soma += nota; // Calcula a soma das notas
    }
    double media = soma / notas.size(); // Calcula a média das notas
    System.out.printf(format:"Média: %.2f\n", media); // Mostra a média com duas casas decimais
}
```

Figura x.: Verifica se foram introduzidas notas válidas para o aluno e, se existirem notas, calcula a soma e a média, apresentando-a formatada com duas casas decimais.

- Fecha o objeto Scanner e liberta os recursos associados:

```
scanner.close(); // Fecha o Scanner para libertar recursos
```

Figura xi.: Fecha o objeto Scanner e liberta os recursos associados:

IMPLEMENTAÇÃO EM PYTHON – CÓDIGO COMPLETO COMENTADO

```
1 def verificar_dados_alunos():
2     """
3     Solicita o nome de um aluno e as suas notas, verificando se as notas são válidas (entre 0 e 20).
4     Armazena os dados num dicionário e exibe as informações verificadas.
5     """
6     nome_aluno = input("Digite o nome do aluno: ") # Pede ao utilizador o nome do aluno
7     notas = [] # Cria uma lista para armazenar as notas
8
9     while True: # Inicia um ciclo infinito para inserir várias notas
10        try:
11            nota_str = input("Digite uma nota do aluno (ou 'fim' para terminar): ") # Pede ao utilizador uma nota
12            if nota_str.lower() == 'fim': # Se o utilizador escrever 'fim', termina o ciclo
13                break
14            nota = float(nota_str) # Tenta converter a nota para float
15            if 0 <= nota <= 20: # Verifica se a nota está no intervalo permitido
16                notas.append(nota) # Adiciona a nota à lista
17            else:
18                print("Erro: A nota deve estar entre 0 e 20.") # Mensagem de erro se a nota for inválida
19        except ValueError:
20            print("Erro: Por favor, digite um número válido para a nota.") # Mensagem de erro se não for possível converter para número
21
22        dados_aluno = {
23            'nome': nome_aluno, # Guarda o nome do aluno
24            'notas': notas # Guarda a lista de notas
25        }
26
27        print("\nDados do aluno verificados:")
28        print(f"Nome: {dados_aluno['nome']}") # Mostra o nome do aluno
29        if dados_aluno['notas']: # Se existirem notas inseridas
30            print(f"Notas: {' '.join(map(str, dados_aluno['notas']))}") # Mostra a lista de notas
31            media = sum(dados_aluno['notas']) / len(dados_aluno['notas']) # Calcula a média das notas
32            print(f"Média: {media:.2f}") # Mostra a média com duas casas decimais
33        else:
34            print("Nenhuma nota foi inserida para este aluno.") # Mensagem caso não tenham sido inseridas notas
35
36        # Executar o algoritmo
37        verificar_dados_alunos()
```

Figura xii.: Print do código act.py comentado.

IMPLEMENTAÇÃO EM PYTHON – EXCERTOS MAIS RELEVANTES

- Solicita e lê do utilizador o nome do aluno, armazenando-o na variável nome_aluno e inicializa uma lista vazia destinada ao armazenamento das notas inseridas:

```
nome_aluno = input("Digite o nome do aluno: ") # Pede ao utilizador o nome do aluno
notas = [] # Cria uma lista para armazenar as notas
```

Figura xiii.: Solicita e lê do utilizador o nome do aluno, armazenando-o na variável nome_aluno e inicializa uma lista vazia destinada ao armazenamento das notas inseridas.

- Inicia um ciclo infinito que permitirá ao utilizador inserir múltiplas notas, terminando apenas quando for digitado "fim":

```
while True: # Inicia um ciclo infinito para inserir várias notas
```

Figura xiv.: Inicia um ciclo infinito que permitirá ao utilizador inserir múltiplas notas, terminando apenas quando for digitado "fim".

- Solicita a nota ao utilizador e verifica se o comando "fim" foi digitado (ignorando maiúsculas/minúsculas) para terminar o ciclo:

```
nota_str = input("Digite uma nota do aluno (ou 'fim' para terminar): ") # Pede ao utilizador uma nota
if nota_str.lower() == 'fim': # Se o utilizador escrever 'fim', termina o ciclo
    break
```

Figura xv.: Solicita a nota ao utilizador e verifica se o comando "fim" foi digitado (ignorando maiúsculas/minúsculas) para terminar o ciclo.

- Conversão da string para valor decimal (float) e validação do intervalo permitido. Se a nota for válida, é adicionada à lista de notas; caso contrário, é apresentada uma mensagem de erro.

```
nota = float(nota_str) # Tenta converter a nota para float
if 0 <= nota <= 20: # Verifica se a nota está no intervalo permitido
    notas.append(nota) # Adiciona a nota à lista
else:
    print("Erro: A nota deve estar entre 0 e 20.") # Mensagem de erro se a nota for inválida
```

Figura xvi.: Conversão da string para valor decimal (float) e validação do intervalo permitido. Se a nota for válida, é adicionada à lista de notas; caso contrário, é apresentada uma mensagem de erro.

- Tratamento de exceção: caso a conversão falhe (por exemplo, caracteres não numéricos), apresenta uma mensagem de erro ao utilizador.

```
except ValueError:
    print("Erro: Por favor, digite um número válido para a nota.") # Mensagem de erro se não for possível converter para número
```

Figura xvii.: Tratamento de exceção: caso a conversão falhe (por exemplo, caracteres não numéricos), apresenta uma mensagem de erro ao utilizador.

- Se existirem notas válidas, apresenta a lista das notas e calcula a média (com duas casas decimais) e se não forem inseridas notas, informa o utilizador.

```
if dados_aluno['notas']: # Se existirem notas inseridas
    print(f"Notas: {', '.join(map(str, dados_aluno['notas']))}") # Mostra a lista de notas
    media = sum(dados_aluno['notas']) / len(dados_aluno['notas']) # Calcula a média das notas
    print(f"Média: {media:.2f}") # Mostra a média com duas casas decimais
else:
    print("Nenhuma nota foi inserida para este aluno.") # Mensagem caso não tenham sido inseridas notas
```

Figura xviii.: Se existirem notas válidas, apresenta a lista das notas e calcula a média (com duas casas decimais) e se não forem inseridas notas, informa o utilizador.

REFLEXÃO SOBRE AS PRINCIPAIS DIFERENÇAS ENTRE O CÓDIGO EM PYTHON

E EM JAVA

A implementação do mesmo algoritmo em Python e em Java evidencia de forma clara as diferenças estruturais, sintáticas e paradigmáticas entre estas duas linguagens de programação. Destacam-se as seguintes diferenças fundamentais:

1. Sintaxe e Verbosidade

- O Python caracteriza-se por uma sintaxe simples, concisa e de fácil leitura. É possível implementar a mesma lógica com menos linhas de código e de forma mais direta, tornando o código mais acessível, sobretudo para iniciantes.
- O Java, por sua vez, exige uma maior verbosidade. A declaração explícita dos tipos de dados, a necessidade de estruturar o código em classes e métodos, e o uso obrigatório de instruções como import e Scanner, tornam o código mais extenso e formalizado.

2. Gestão de Tipos de Dados

- O Java é uma linguagem fortemente tipada, obrigando à declaração prévia do tipo de cada variável (String, double, etc.), o que reduz o risco de erros em tempo de execução, mas exige maior rigor na escrita do código.

- Em Python, a tipagem é dinâmica, permitindo maior flexibilidade e rapidez na definição de variáveis, mas podendo originar erros se não houver atenção à coerência dos tipos.

3. Estruturas de Dados

- Em Java, a utilização de uma `ArrayList<Double>` é necessária para armazenar as notas, enquanto em Python uma simples lista (`list`) é suficiente, tornando o código mais direto e menos dependente de estruturas externas.
- O acesso e manipulação de listas em Python é também mais intuitivo e com menos código associado.

4. Leitura de Dados do Utilizador

- O Java utiliza o objeto `Scanner` para leitura da entrada, que exige a criação e posterior encerramento do mesmo (`scanner.close()`), sob pena de má gestão de recursos.
- Em Python, a função `input()` é suficiente para recolher dados do utilizador, sem necessidade de criação de objetos ou gestão de recursos adicionais.

5. Tratamento de Exceções

- Ambas as linguagens permitem a captura e tratamento de exceções, embora com sintaxe diferente: `try-catch` em Java e `try-except` em Python. Em Python, este mecanismo é mais compacto e integrado na própria estrutura do ciclo.
- Em Java, o tratamento de exceções implica a criação explícita do bloco `try-catch`, indicando claramente o tipo de exceção a capturar.

6. Estrutura Geral do Programa

- O Java obriga à existência de uma estrutura de classes e de um método principal (`public static void main`), reforçando o paradigma orientado a objetos.
- O Python permite a execução direta de funções, sem necessidade de estruturação em classes, facilitando a prototipagem e experimentação.

7. Apresentação dos Resultados

- Em Java, a apresentação da média é feita através do método `System.out.printf`, com formatação explícita.
- Em Python, a formatação da média é obtida de forma simples através de f-strings.

8. Paradigma de Programação

- Java está claramente orientado ao paradigma orientado a objetos, mesmo para algoritmos simples, ao passo que Python permite um paradigma mais procedural e flexível.

Em suma:

O Python oferece maior simplicidade, rapidez e flexibilidade para pequenas aplicações e protótipos, sendo muito apreciado pela sua curva de aprendizagem reduzida e elevada legibilidade.

O Java, por outro lado, proporciona maior robustez, controlo e estrutura, sendo preferido em projetos de grande escala, onde a gestão rigorosa dos tipos e dos recursos é fundamental.

LIMITAÇÕES DO ALGORITMO

Apesar de cumprir os requisitos mínimos do enunciado, o algoritmo apresentado, em ambas as versões (Java e Python), apresenta algumas limitações relevantes que importa salientar:

1. Apenas Um Aluno por Execução

- O algoritmo foi concebido para registar apenas um aluno de cada vez. Não permite o armazenamento, consulta ou processamento de dados relativos a múltiplos alunos numa mesma execução, limitando a sua utilidade em contextos educativos reais.

2. Falta de Persistência de Dados

- Os dados recolhidos (nome e notas) apenas existem enquanto o programa está a ser executado, não sendo guardados em qualquer ficheiro, base de dados ou sistema de armazenamento permanente. Isso inviabiliza consultas futuras ou a reutilização dos dados.

3. Interface de Utilizador Limitada

- A interação é realizada exclusivamente através da linha de comandos (CLI), não oferecendo uma interface gráfica ou uma experiência de utilização mais intuitiva, ou acessível a utilizadores menos experientes.

4. Validação de Dados Simples

- A validação dos dados limita-se a garantir que as notas estão entre 0 e 20 e que o valor inserido é numérico. Não existe validação para outros aspetos, como a garantia de que o nome do aluno não está vazio, ou o controlo sobre o número de casas decimais nas notas.

5. Ausência de Funcionalidades Adicionais

- O algoritmo não permite a edição ou remoção de notas após a sua inserção, nem oferece funcionalidades como o cálculo de estatísticas adicionais (ex: nota máxima, nota mínima) ou a exportação dos resultados.

6. Escalabilidade e Manutenção

- Sendo um algoritmo simples e linear, não está preparado para ser expandido ou integrado facilmente em sistemas maiores, carecendo de modularização e separação clara de responsabilidades.

7. Dependência do Utilizador

- Toda a validação é reativa, ou seja, o algoritmo apenas deteta e informa o utilizador quando ocorre um erro, não prevenindo, por exemplo, a inserção repetida do mesmo valor ou entradas desnecessárias.

MELHORIAS QUE PODIAM SER IMPLEMENTADAS

Com vista à evolução do algoritmo e à sua adaptação a cenários mais exigentes ou realistas, identificam-se várias melhorias possíveis:

1. Permitir o Registo de Vários Alunos

- Uma das principais evoluções passaria pela implementação de um mecanismo que permitisse a inserção e armazenamento dos dados de múltiplos alunos numa só execução, possibilitando o cálculo e apresentação de médias individuais e gerais.

2. Implementação de Persistência de Dados

- A introdução de funcionalidades para guardar os dados em ficheiros (texto, CSV, JSON) ou bases de dados permitiria a consulta posterior, o processamento estatístico e a recuperação dos registos em execuções futuras.

3. Melhoria da Interface de Utilizador

- O desenvolvimento de uma interface gráfica (GUI) tornaria o algoritmo mais acessível a utilizadores menos experientes e melhoraria a experiência de utilização.

4. Validação Adicional de Dados

- Poderia ser reforçada a validação dos dados inseridos, assegurando, por exemplo, que o nome do aluno não é deixado em branco, que não são inseridas notas duplicadas, e limitando o número de casas decimais permitidas nas notas

5. Funcionalidades Estatísticas

- A implementação de novas funcionalidades, como o cálculo da nota máxima, nota mínima, desvio padrão, ou a classificação automática dos resultados, enriqueceria significativamente o algoritmo.

6. Permitir Edição e Remoção de Notas

- Seria útil permitir ao utilizador editar ou remover notas já inseridas antes do cálculo final da média, aumentando a flexibilidade do sistema.

7. Exportação e Impressão de Resultados

- Adicionar a possibilidade de exportar os resultados para formatos populares (PDF, Excel) ou permitir a impressão direta facilitaria a partilha e arquivo dos dados.

8. Modularização do Código

- Reestruturar o código de modo a torná-lo mais modular, separando claramente as funções de entrada de dados, validação, processamento e apresentação, melhoraria a sua legibilidade, manutenção e possibilidade de reutilização.

CONCLUSÃO

O presente trabalho, proposto no âmbito da unidade curricular de Linguagens e Paradigmas de Programação da Licenciatura em Engenharia Informática, permitiu aprofundar a compreensão das diferenças e semelhanças entre as linguagens de programação Java e Python, através da análise comparativa da implementação de um algoritmo simples, mas representativo dos conceitos fundamentais da programação.

Ao longo do relatório, foi possível constatar que ambas as linguagens conseguem responder eficazmente ao problema proposto, ainda que recorrendo a abordagens distintas em termos de sintaxe, estrutura e paradigma. O Python destacou-se pela sua simplicidade, concisão e facilidade de leitura, características que o tornam particularmente atrativo para tarefas de desenvolvimento rápido e prototipagem. Por outro lado, o Java evidenciou-se pela sua robustez, rigor na gestão de tipos e orientação a objetos, sendo mais indicado para contextos em que a escalabilidade, a manutenção e a estruturação do código assumem especial importância.

Foram identificadas limitações relevantes no algoritmo, nomeadamente a impossibilidade de registar múltiplos alunos, a ausência de mecanismos de persistência de dados e a interface restrita à linha de comandos. Paralelamente, foram propostas várias melhorias, que, a serem implementadas, permitiriam uma aplicação mais robusta, flexível e próxima das necessidades reais dos utilizadores.

A realização deste exercício contribuiu não só para consolidar competências técnicas em programação, mas também para o desenvolvimento do espírito crítico relativamente à escolha das ferramentas e abordagens mais adequadas em cada contexto. Reforça-se, assim, a importância de uma análise cuidada das necessidades e restrições do problema antes da seleção da linguagem ou do paradigma de programação a adotar, valorizando o papel da reflexão fundamentada na formação do engenheiro informático.

ANEXOS

```
teste.java > ...
1  package com.mycompany.java_project;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Scanner;
6
7  public class teste {
8
9      Run | Debug
10     public static void main(String[] args) {
11
12         Scanner scanner = new Scanner(System.in); // Cria um objeto Scanner para ler a entrada do utilizador
13
14         System.out.print(s:"Digite o nome do aluno: ");
15         String nomeAluno = scanner.nextLine(); // Lê o nome do aluno inserido pelo utilizador
16
17         List<Double> notas = new ArrayList<>(); // Cria uma lista para armazenar as notas do aluno
18
19         while (true) { // Inicia um ciclo infinito para inserir várias notas
20             System.out.print(s:"Digite uma nota do aluno (ou 'fim' para terminar): ");
21             String notaStr = scanner.nextLine(); // Lê a nota inserida pelo utilizador como String
22
23             if (notaStr.equalsIgnoreCase(anotherString:"fim")) { // Se o utilizador escrever 'fim', termina o ciclo
24                 break;
25             }
26
27             try {
28                 double nota = Double.parseDouble(notaStr); // Tenta converter a nota para double
29                 if (nota >= 0 && nota <= 20) { // Verifica se a nota está no intervalo permitido
30                     notas.add(nota); // Adiciona a nota à lista
31                 } else {
32                     System.out.println(x:"Erro: A nota deve estar entre 0 e 20."); // Mensagem de erro se a nota for
33                                     // inválida
34                 }
35             } catch (NumberFormatException e) {
36                 System.out.println(x:"Erro: Por favor, digite um número válido para a nota."); // Mensagem de erro se não
37                                     // for possível converter
38                                     // para número
39             }
40         }
41
42         System.out.println(x:"\nDados do aluno verificados:");
43         System.out.println("Nome: " + nomeAluno); // Mostra o nome do aluno
44
45         if (!notas.isEmpty()) { // Se existirem notas inseridas
46             System.out.println("Notas: " + notas); // Mostra a lista de notas
47             double soma = 0;
48             for (double nota : notas) {
49                 soma += nota; // Calcula a soma das notas
50             }
51             double media = soma / notas.size(); // Calcula a média das notas
52             System.out.printf(format:"Média: %.2f\n", media); // Mostra a média com duas casas decimais
53         } else {
54             System.out.println(x:"Nenhuma nota foi inserida para este aluno."); // Mensagem caso não tenham sido inseridas
55                                     // notas
56         }
57
58         scanner.close(); // Fecha o Scanner para libertar recursos
59     }
60 }
```

```
1 def verificar_dados_alunos():
2     """
3     Solicita o nome de um aluno e as suas notas, verificando se as notas são válidas (entre 0 e 20).
4     Armazena os dados num dicionário e exibe as informações verificadas.
5     """
6     nome_aluno = input("Digite o nome do aluno: ") # Pede ao utilizador o nome do aluno
7     notas = [] # Cria uma lista para armazenar as notas
8
9     while True: # Inicia um ciclo infinito para inserir várias notas
10        try:
11            nota_str = input("Digite uma nota do aluno (ou 'fim' para terminar): ") # Pede ao utilizador uma nota
12            if nota_str.lower() == 'fim': # Se o utilizador escrever 'fim', termina o ciclo
13                break
14            nota = float(nota_str) # Tenta converter a nota para float
15            if 0 <= nota <= 20: # Verifica se a nota está no intervalo permitido
16                notas.append(nota) # Adiciona a nota à lista
17            else:
18                print("Erro: A nota deve estar entre 0 e 20.") # Mensagem de erro se a nota for inválida
19        except ValueError:
20            print("Erro: Por favor, digite um número válido para a nota.") # Mensagem de erro se não for possível converter para número
21
22        dados_aluno = {
23            'nome': nome_aluno, # Guarda o nome do aluno
24            'notas': notas # Guarda a lista de notas
25        }
26
27        print("\nDados do aluno verificados:")
28        print(f"Nome: {dados_aluno['nome']}") # Mostra o nome do aluno
29        if dados_aluno['notas']: # Se existirem notas inseridas
30            print(f"Notas: {', '.join(map(str, dados_aluno['notas']))}") # Mostra a lista de notas
31            media = sum(dados_aluno['notas']) / len(dados_aluno['notas']) # Calcula a média das notas
32            print(f"Média: {media:.2f}") # Mostra a média com duas casas decimais
33        else:
34            print("Nenhuma nota foi inserida para este aluno.") # Mensagem caso não tenham sido inseridas notas
35
36 # Executar o algoritmo
37 verificar_dados_alunos()
```